Height-Resolved Curvature Decomposition: Separating Real Stability Curvature from Coordinate Effects

David E. England, PhD (dee0001@uah.edu) and the UAH BL Team

The 2D contour diagnostic panel demonstrates that sharp vertical profile features near the critical Richardson number ($Ri_c \approx 0.20$) often stem from coordinate stretching rather than purely local thermodynamic phase changes. When turbulent flux varies with height, the local Obukhov length scale $L(z)$ distorts the vertical coordinate frame.

We decompose the total profile curvature of $Ri(z)$ into distinct physical and numerical components:

$$Ri_{zz} = \underbrace{Ri_{\zeta\zeta}\zeta_z^2}_{C_{\text{const}}} + \underbrace{Ri_{\zeta}\zeta_{zz}}_{C_{\text{coord}}} + E_{\text{error}}$$

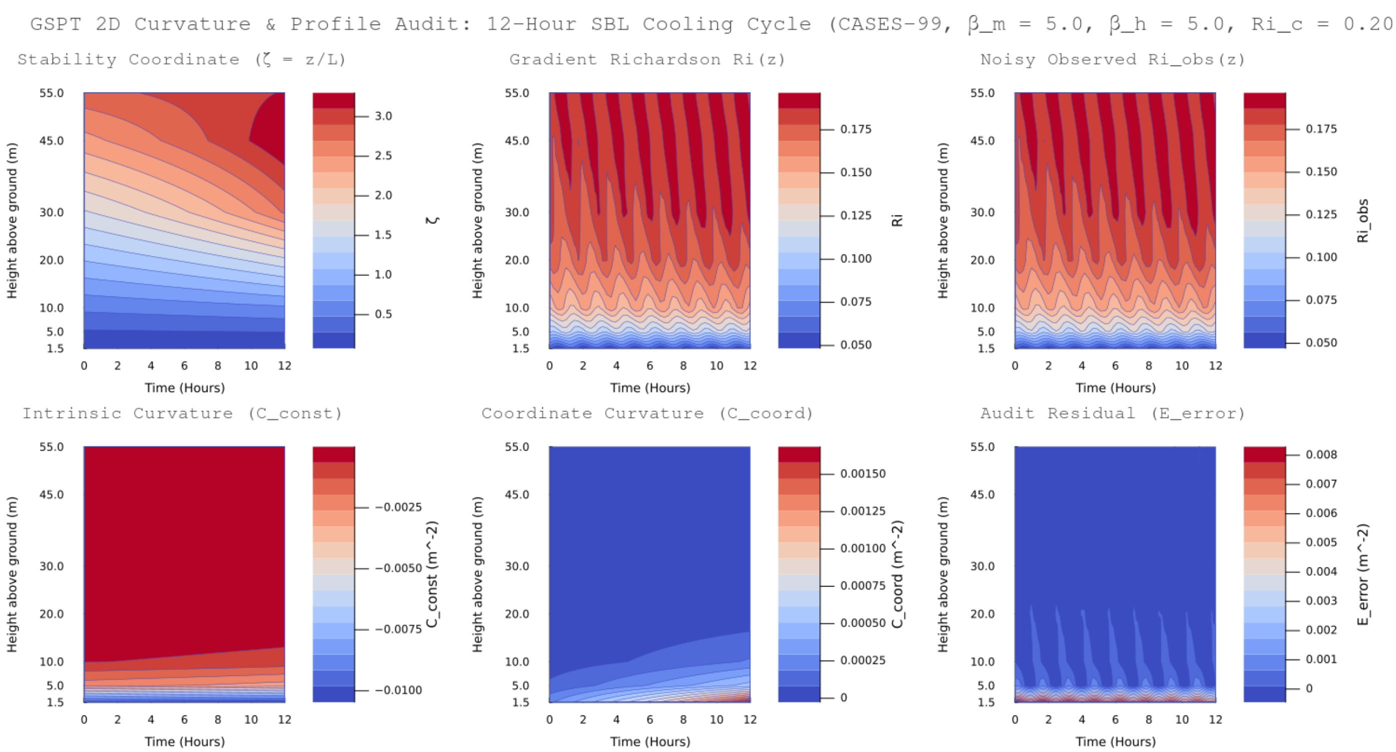
- **Intrinsic Stability Curvature (C_{const}):** Expected curvature from local thermodynamic stability holding the similarity coordinate fixed. Under Monin–Obukhov similarity ($\beta_m = 5.0, \beta_h = 5.0$), this term remains negative, reflecting gradual lower-level stabilization.
- **Coordinate-Stretching Curvature (C_{coord}):** Curvature introduced because

$\zeta(z) = z / L(z)$ varies with height under flux-divergent SBL conditions. Near nocturnal low-level jet (LLJ) noses, this coordinate effect spikes strongly positive, counteracting intrinsic cooling.

- **Audit Residual (E_{error}):** Quantifies operator truncation error ($\mathbf{D}*2 Ri*{\text{smooth}} - Ri_{\text{exact}}$). Keeping numerical errors separate prevents finite-difference artifacts on non-uniform CASES-99 tower grids (1.5 m to 55 m) from contaminating physical curvature fields.

Key Physical Insights (12-Hour SBL Cooling Cycle)

- **Near-Surface Cancellation ($z \approx 10\ \mathrm{m}$):** During deep stable hours (Hours 6–12), positive coordinate curvature ($C_{\text{coord}} \approx +0.0036\ \mathrm{m}^{-2}$) cancels negative stability curvature ($C_{\text{const}} \approx -0.0040\ \mathrm{m}^{-2}$). The resulting locally linear profile ($Ri_{zz} \approx -0.0004\ \mathrm{m}^{-2}$) hides active, offsetting physical processes.
- **Noise Suppression & Filtering:** Synthetic sensor jitter ($\sigma_{\text{obs}} = 0.008$) added to Ri_{obs} is smoothed via Tikhonov-regularized differentiation ($\lambda_{\text{reg}} = 5.0$) before operator evaluation, isolating physical signal from observational noise.



6-Panel Diagnostic Layout

Panel Position	Variable	Physical / Numerical Interpretation
Top Left	Stability Coordinate (ζ)	Dimensionless height ratio $z / L(z)$ across time and altitude.
Top Center	Gradient Richardson (Ri)	Exact analytical profile derived from SBL similarity theory.
Top Right	Observed Richardson (Ri_{obs})	Synthetic sensor data containing Gaussian observational jitter.
Bottom Left	Intrinsic Curvature (C_{const})	Pure thermodynamic curvature assuming constant coordinate scaling.
Bottom Center	Coordinate Curvature (C_{coord})	Curvature generated by vertical flux divergence and LLJ shear.
Bottom Right	Audit Residual (E_{error})	Explicit finite-difference discretization residual (\$\mathbf{D}2 *Ri*{\text{smooth}} - Ri_{\text{exact}}\$).

Code:

```
#!/usr/bin/env julia
# src/gspt_curvature_contour_v3.jl
# version 3 of the GSPT curvature contour plotting script
# David E. England, Ph.D.
# email: david.england@uah.edu
using LinearAlgebra
using Printf

# Try to use Plots.jl
try
    using Plots
catch e
    @warn "Plots.jl package not found in current environment. Script wi
end
```

```julia
function stencil_weights(z_stencil::Vector{Float64}, z0::Float64, m::Ir
    p = length(z_stencil)
    A = [Float64(z - z0)^(k - 1) / factorial(k - 1) for k in 1:p, z in
    b = zeros(p)
    b[m + 1] = 1.0  # Select target derivative order
    return A \ b
end

function build_operators(z::Vector{Float64})
    n = length(z)
    D1 = zeros(n, n)
    D2 = zeros(n, n)

    for i in 1:n
        if i == 1
            idx = [1, 2, 3]
        elseif i == n
            idx = [n-2, n-1, n]
        else
            idx = [i-1, i, i+1]
        end

        D1[i, idx] = stencil_weights(z[idx], z[i], 1)
        D2[i, idx] = stencil_weights(z[idx], z[i], 2)
    end

    return D1, D2
end

Ri_model(ζ, β_m, β_h) = ζ * (1.0 + β_h * ζ) / (1.0 + β_m * ζ)^2

function Ri_zeta(ζ, β_m, β_h)
    return (1.0 + (2.0 * β_h - β_m) * ζ) / (1.0 + β_m * ζ)^3
end

function Ri_zetazeta(ζ, β_m, β_h)
    num = 2.0 * β_h - 4.0 * β_m + 2.0 * β_m * (β_m - 2.0 * β_h) * ζ
    return num / (1.0 + β_m * ζ)^4
end

function plot_gspt_curvature_layers(
    hours::Vector{Float64},
    z_tower::Vector{Float64},
    zeta_mat::Matrix{Float64},
    ri_smooth_mat::Matrix{Float64},
    ri_obs_mat::Matrix{Float64},
    C_const::Matrix{Float64},
```

```julia
    C_coord::Matrix{Float64},
    E_error::Matrix{Float64};
    save_path::String = "gspt_curvature_contour_v3.png"
)
    if !@isdefined Plots
        @warn "Plots.jl not loaded. Skipping PNG rendering."
        return nothing
    end

    gr() # Set GR backend

    # Common plotting parameters
    plt_opts = (
        xlabel = "Time (Hours)",
        ylabel = "Height above ground (m)",
        yticks = (z_tower, [@sprintf("%.1f", h) for h in z_tower]),
        fill = true,
        c = :coolwarm,
        linewidth = 0.5,
        tickfont = font(8, "sans-serif"),
        guidefont = font(9, "sans-serif"),
        titlefont = font(10, "sans-serif bold")
    )

    # Row 1 plots
    p1 = contourf(hours, z_tower, zeta_mat;
                  title="Stability Coordinate (ζ = z/L)",
                  colorbar_title="ζ",
                  plt_opts...)

    p2 = contourf(hours, z_tower, ri_smooth_mat;
                  title="Gradient Richardson Ri(z)",
                  colorbar_title="Ri",
                  plt_opts...)

    p3 = contourf(hours, z_tower, ri_obs_mat;
                  title="Noisy Observed Ri_obs(z)",
                  colorbar_title="Ri_obs",
                  plt_opts...)

    # Row 2 plots
    p4 = contourf(hours, z_tower, C_const;
                  title="Intrinsic Curvature (C_const)",
                  colorbar_title="C_const (m^-2)",
                  plt_opts...)

    p5 = contourf(hours, z_tower, C_coord;
                  title="Coordinate Curvature (C_coord)",
```

```
                 colorbar_title="C_coord (m^-2)",
                 plt_opts...)

    p6 = contourf(hours, z_tower, E_error;
                 title="Audit Residual (E_error)",
                 colorbar_title="E_error (m^-2)",
                 plt_opts...)

    # Assemble into a 2x3 diagnostic panel layout
    full_plot = plot(p1, p2, p3, p4, p5, p6,
                    layout = (2, 3),
                    size = (1500, 800),
                    plot_title = "GSPT 2D Curvature & Profile Audit: 1
                    plot_titlefont = font(12, "sans-serif bold"),
                    margin = 6Plots.mm)

    savefig(full_plot, save_path)
    println("Successfully rendered and saved: $save_path")
    return full_plot
end

function run_test_simulation()
    println("Setting up SCM 12-hour cooling cycle...")

    # CASES-99 55m Tower heights
    z_tower = [1.5, 5.0, 10.0, 20.0, 30.0, 45.0, 55.0]
    n_z = length(z_tower)
    D1, D2 = build_operators(z_tower)

    # Time steps: 50 points over 12 hours
    hours = collect(range(0.0, 12.0, length=50))
    n_t = length(hours)

    # Curvature & profile matrices
    zeta_mat = zeros(n_z, n_t)
    ri_smooth_mat = zeros(n_z, n_t)
    ri_obs_mat = zeros(n_z, n_t)
    C_const_mat = zeros(n_z, n_t)
    C_coord_mat = zeros(n_z, n_t)
    E_error_mat = zeros(n_z, n_t)

    # SBL model settings (using user-requested grassland parameters)
    β_m, β_h = 5.0, 5.0
    L0 = 20.0
    sigma_obs = 0.008
    lambda_reg = 5.0

    for t_idx in 1:n_t
```

```
        t = hours[t_idx]

        # Non-stationary LLJ Jet nose intensity growing with cooling
        amp = 0.1 + 0.35 * (t / 12.0)
        L_profile = [L0 * (1.0 - amp * sin(π * zi / 60.0)) for zi in z_

        # Similarity coordinate profile
        ζ = z_tower ./ L_profile
        ζ_z = D1 * ζ
        ζ_zz = D2 * ζ

        zeta_mat[:, t_idx] = ζ

        # Analytical GSPT curvature
        ri_z = [Ri_zeta(ζ[i], β_m, β_h) for i in 1:n_z]
        ri_zz = [Ri_zetazeta(ζ[i], β_m, β_h) for i in 1:n_z]

        C_const = ri_zz .* (ζ_z .^ 2)
        C_coord = ri_z .* ζ_zz
        Ri_exact = C_const .+ C_coord

        C_const_mat[:, t_idx] = C_const
        C_coord_mat[:, t_idx] = C_coord

        # Add random sensor jitter (using deterministic seed offsets fc
        noise = sigma_obs .* sin.(t_idx .+ (1:n_z) .* 1.5)
        Ri_obs = [Ri_model(ζ[i], β_m, β_h) for i in 1:n_z] .+ noise
        ri_obs_mat[:, t_idx] = Ri_obs

        # Track A: Tikhonov regularization filter
        R = D2' * D2
        A_reg = I(n_z) + lambda_reg .* R
        Ri_smooth = A_reg \ Ri_obs
        ri_smooth_mat[:, t_idx] = Ri_smooth

        # Computed physical curvature
        M_Ri_zz = D2 * Ri_smooth

        # Discrete audit error
        E_error_mat[:, t_idx] = M_Ri_zz .- Ri_exact
    end

    println("Matrices populated successfully. Generating Plots panel...

    plot_gspt_curvature_layers(hours, z_tower, zeta_mat, ri_smooth_mat,
                               C_const_mat, C_coord_mat, E_error_mat;
                               save_path = "gspt_curvature_contour_v3.p
end
```

The script sets up a complete synthetic boundary layer experiment designed to evaluate how numerical operators handle non-uniform tower grids, flux-divergent jet dynamics, observational jitter, and spatial filtering.

1. Non-Uniform Stencil Differentiation ($\mathbf{D}_1, \mathbf{D}_2$)

- **Method:** Computes derivative weights by solving a local Taylor expansion system $\mathbf{A}\,\mathbf{w} = \mathbf{b}$ for derivative orders $m = 1$ ($\mathbf{D}_1$) and $m = 2$ ($\mathbf{D}_2$).
- **Grid Adaptation:** Because CASES-99 tower levels are irregularly spaced, ranging from $1.5\ \mathrm{m}$ near the surface to $10\ \mathrm{m}$ spacing aloft, standard uniform finite differences introduce severe errors. The code applies 3-point centered stencils on interior nodes and one-sided 3-point stencils at the top and bottom boundaries.

2. Synthetic Low-Level Jet (LLJ) Signature

- **Mechanism:** Prescribes a synthetic Obukhov length field $L(z, t)$ whose vertical structure mimics the stability signature associated with a developing nocturnal LLJ, modulated over the 12-hour cooling period:

$$L(z,t) = L_0\left[1 - \left(0.1 + 0.35\frac{t}{12}\right)\sin\left(\frac{\pi z}{60}\right)\right]$$

- **Physical Effect:** Produces a $Ri(z)$ profile with the curvature signature typical of a developing nocturnal LLJ nose around $z \approx 30\text{–}45\ \mathrm{m}$. This introduces vertical flux divergence ($\zeta_{zz} \neq 0$), which generates the positive coordinate curvature (C_{coord}) that competes with surface cooling (C_{const}).

3. Observational Noise Jitter

- **Formula:** $Ri_{\mathrm{obs}}(z,t) = Ri_{\mathrm{exact}}(z,t) + \sigma_{\mathrm{obs}}\sin(t_{\mathrm{idx}} + 1.5z_{\mathrm{idx}})$
- **Purpose:** Injects continuous sensor noise ($\sigma_{\mathrm{obs}} = 0.008$) into exact similarity fields. Using deterministic sine offsets mimics high-frequency instrument jitter while keeping the benchmark strictly reproducible across runs.

4. Tikhonov Regularization (Smoothing)

- **Operator:** Solves the linear system $(\mathbf{I} + \lambda_{\text{reg}} \mathbf{D}_2^T \mathbf{D}_2) Ri_{\text{smooth}} = Ri_{\text{obs}}$ using regularization parameter $\lambda_{\text{reg}} = 5.0$.
- **Function:** Taking raw numerical derivatives of noisy data catastrophically amplifies high-frequency jitter. The curvature penalty term ($\mathbf{D}_2^T \mathbf{D}_2$) smooths Ri_{obs} prior to second differentiation, allowing the audit residual E_{error} to isolate discretization errors rather than sensor noise.

Quick Parameter Tuning Guide

- **Change Tower Levels (`z_tower`):** Modify the values inside `[1.5, 5.0, ...]` to match actual sensor heights (e.g., add `70.0` or `100.0` for taller boundary layer masts).
- **Adjust SBL Regimes (`beta_m` , `beta_h`):** Set `beta_m = 4.7` and `beta_h = 4.7` for standard Businger–Dyer formulations, lower to `4.0` for weakly stable conditions agreeing with a classical critical value of `0.25` , or increase to `5.0` or more for strongly stable nocturnal conditions.
- **Control Noise Jitter (`sigma_obs`):** Increase `sigma_obs` to `0.02` to test high-noise sonic anemometer environments.
- **Filter Jitter Amplification (`lambda_reg`):** If derivative fields look noisy, increase `lambda_reg` (e.g., `10.0 – 20.0`). If the filter oversmooths physical gradients, reduce it toward `1.0` .